No $tress

Documentação completa – Etapa 3
Programação Orientada a Objetos – Universidade La Salle
Vinícius Quevedo Machado

1. Proposta (revisada)

Problema. Muitas pessoas pagam assinaturas e contas recorrentes (streaming, jogos, nuvem, academia, softwares) e perdem o controle de quais possuem, quando vencem, quando um teste gratuito vira cobrança e quanto tudo isso pesa no mês. O resultado são cobranças esquecidas, um gasto que ninguém enxerga e a clássica pergunta “Pra onde foi meu salário?”.

Para quem. Pessoas físicas, em especial estudantes e jovens profissionais, que hoje controlam isso de cabeça ou em uma planilha confusa.

Solução. O *No $tress* é um sistema desktop em Python orientado a objetos que centraliza as assinaturas, calcula o custo mensal e a projeção anual, avisa dos vencimentos e fins de teste e gera relatórios e gráfico por categoria.

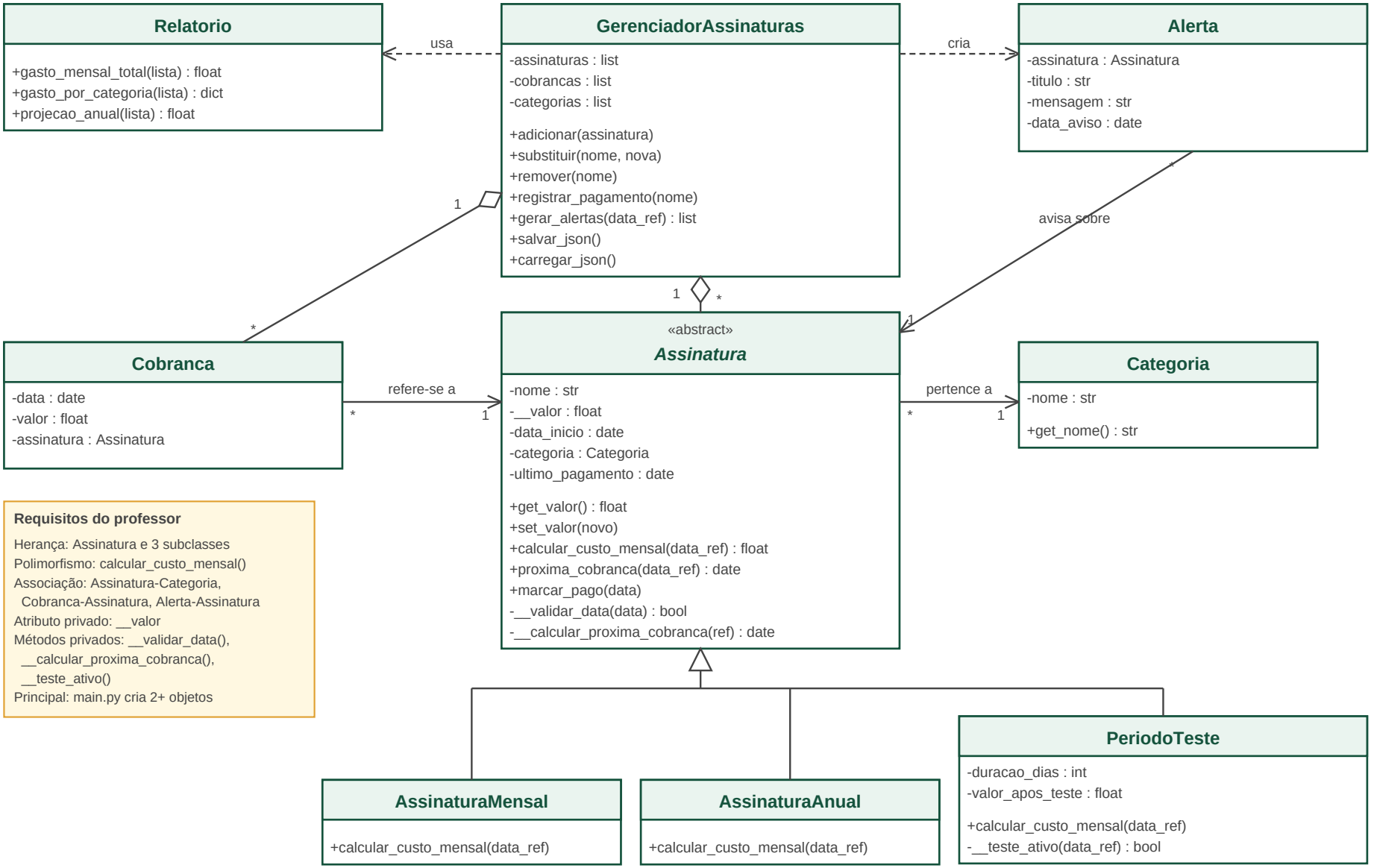
Ferramentas utilizadas

- **Python 3**: linguagem do projeto.
- **VS Code**: editor de código.
- **Tkinter**: interface gráfica (já acompanha o Python).
- **matplotlib**: gráfico de gastos, embutido na janela do Tkinter.
- **unittest**: testes automáticos do domínio (biblioteca padrão).
- **Git e GitHub**: repositório remoto do código (ver seção 5).
- **ReportLab**: geração dos diagramas UML e deste PDF.

O que mudou desde a proposta

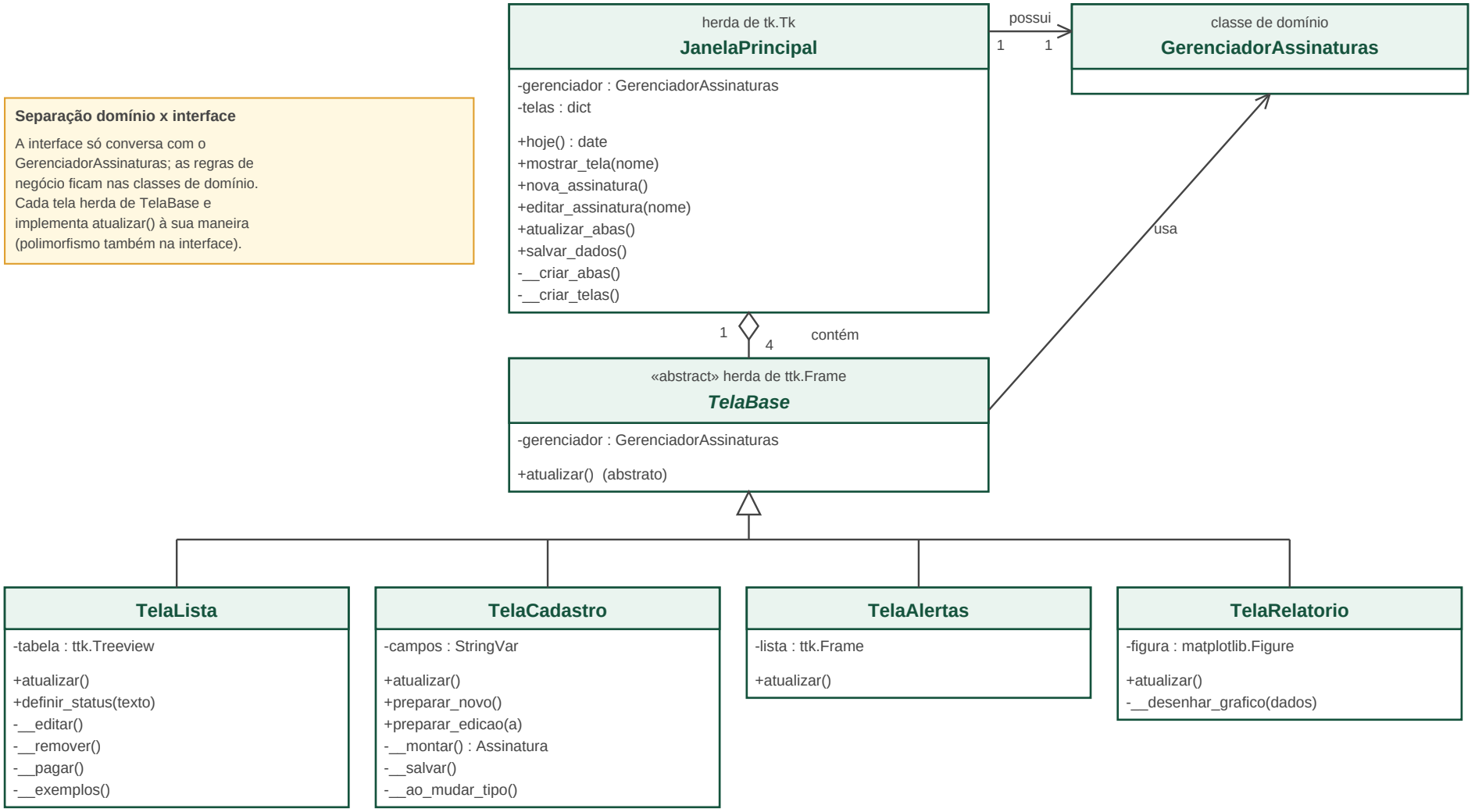
Mudança	Motivo
Interface com 6 classes próprias (JanelaPrincipal, TelaBase e 4 telas); total de 15 classes	Manter 100% da interface em classes e separar domínio de interface.
Cobranças registradas pelo usuário (botão “Registrar pagamento”)	Evita gerar pagamentos que o usuário não confirmou.
Assinatura guarda o último vencimento pago	Para a próxima cobrança avançar depois de um pagamento.
Projeção anual soma os próximos 12 meses, um a um	Testes gratuitos que terminam passam a pesar no resultado.
Alerta ganhou título; Gerenciador ganhou lista de categorias e *substituir()*	Necessários para os alertas e para a edição.
Código publicado no GitHub	O repositório remoto reúne o código, os testes e o README (ver seção 5).

2. Diagramas UML (atualizados) – classes de domínio



Legenda: triângulo vazio = herança; losango vazio = agregação; seta aberta = associação; seta tracejada = dependência (usa/cria); + público, - privado; * = muitos, 1 = um. Diagrama simplificado, com os membros principais.

Diagramas UML – classes da interface gráfica (Tkinter)



Separação domínio x interface

A interface só conversa com o GerenciadorAssinaturas; as regras de negócio ficam nas classes de domínio. Cada tela herda de TelaBase e implementa atualizar() à sua maneira (polimorfismo também na interface).

O programa principal (*main()* em *main.py*) cria o *GerenciadorAssinaturas* e a *JanelaPrincipal* (dois objetos) e inicia a interface.

3. Telas do projeto (prints reais)

Todas as telas abaixo são capturas do programa em funcionamento (*main.py* do repositório), tiradas em uma mesma sessão, com os dados de exemplo e a data de hoje (05/11/2026).

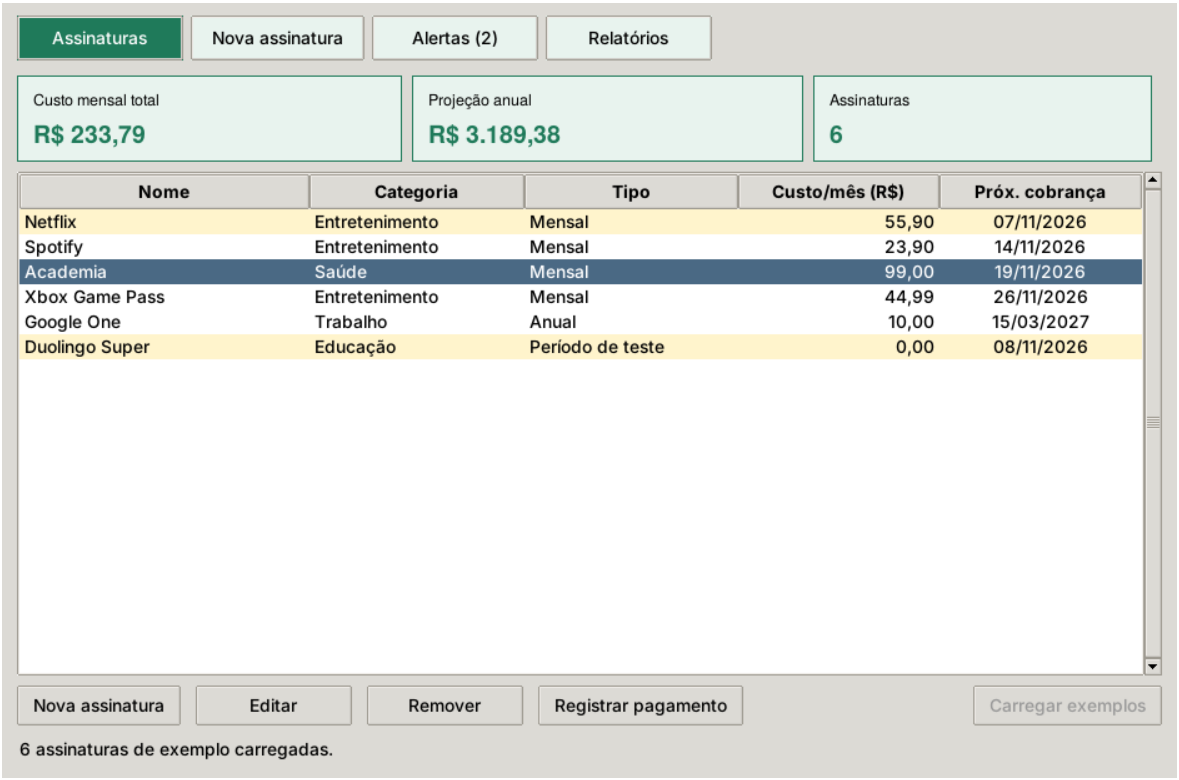


Figura 1 – Tela principal. Mostra o custo mensal total, a projeção anual e a quantidade de assinaturas. A tabela lista cada assinatura com categoria, tipo, custo no mês e próxima cobrança; linhas em amarelo vencem em até 7 dias. Os botões fazem cadastro, edição, remoção e registro de pagamento.

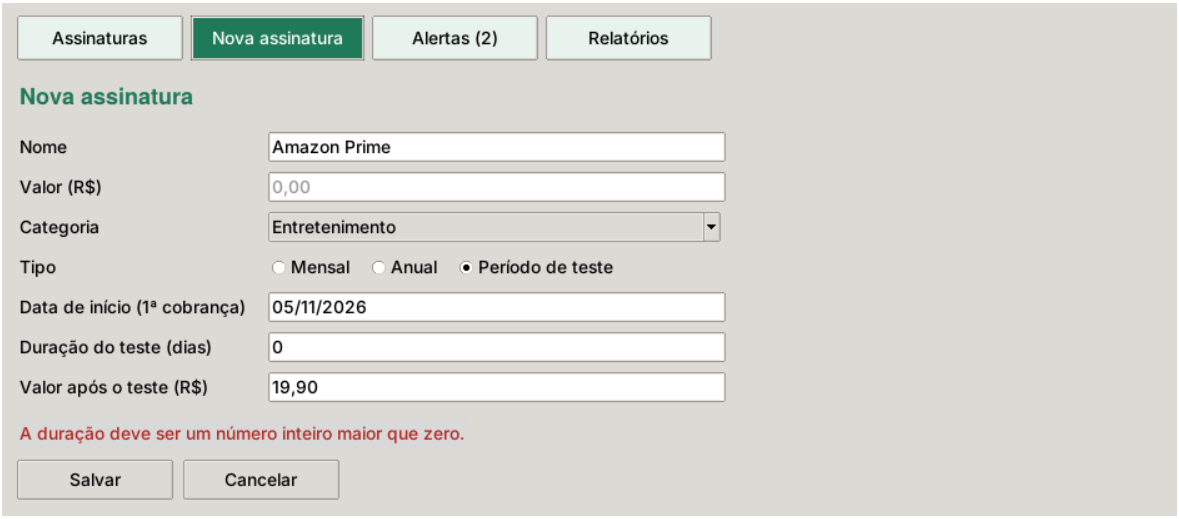


Figura 2 – Cadastro com mensagem de erro. Formulário de nova assinatura. Ao escolher “Período de teste”, o valor fica fixo em 0,00 e aparecem os campos de duração e de valor após o teste. Aqui a duração 0 foi recusada com uma mensagem, e o formulário é mantido.

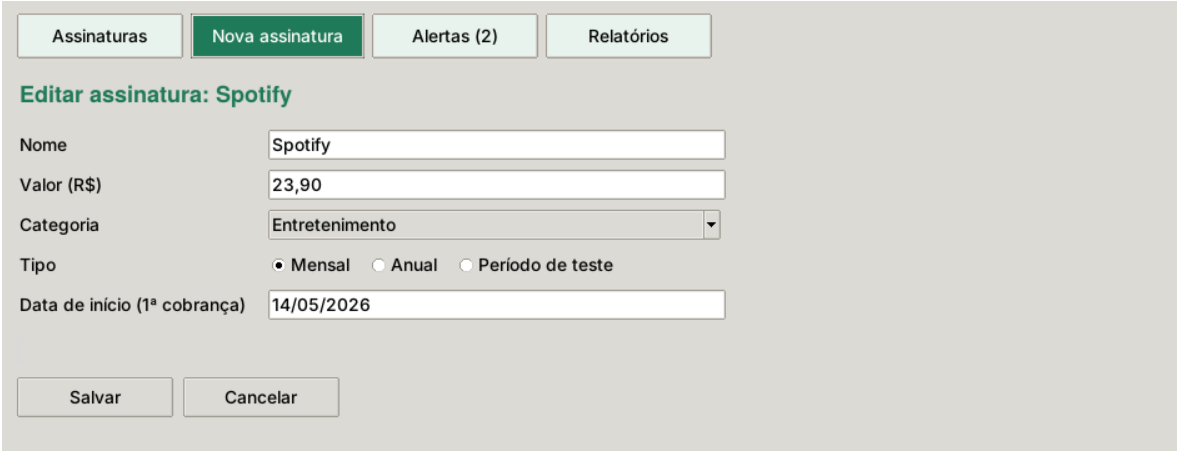


Figura 3 – Edição de assinatura. O mesmo formulário, preenchido com os dados da assinatura escolhida (Spotify). Ao salvar, a assinatura é substituída e o histórico de pagamentos é mantido.

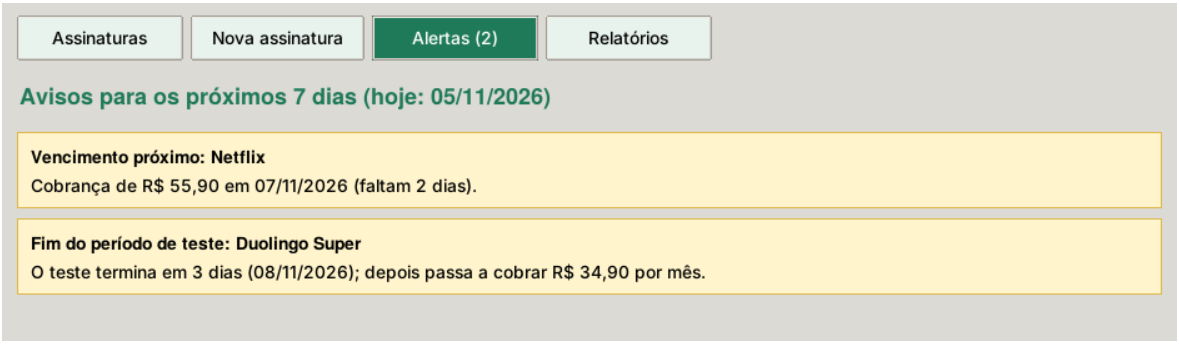


Figura 4 – Alertas. Vencimentos e fins de teste dos próximos 7 dias. O número de alertas aparece na própria aba.

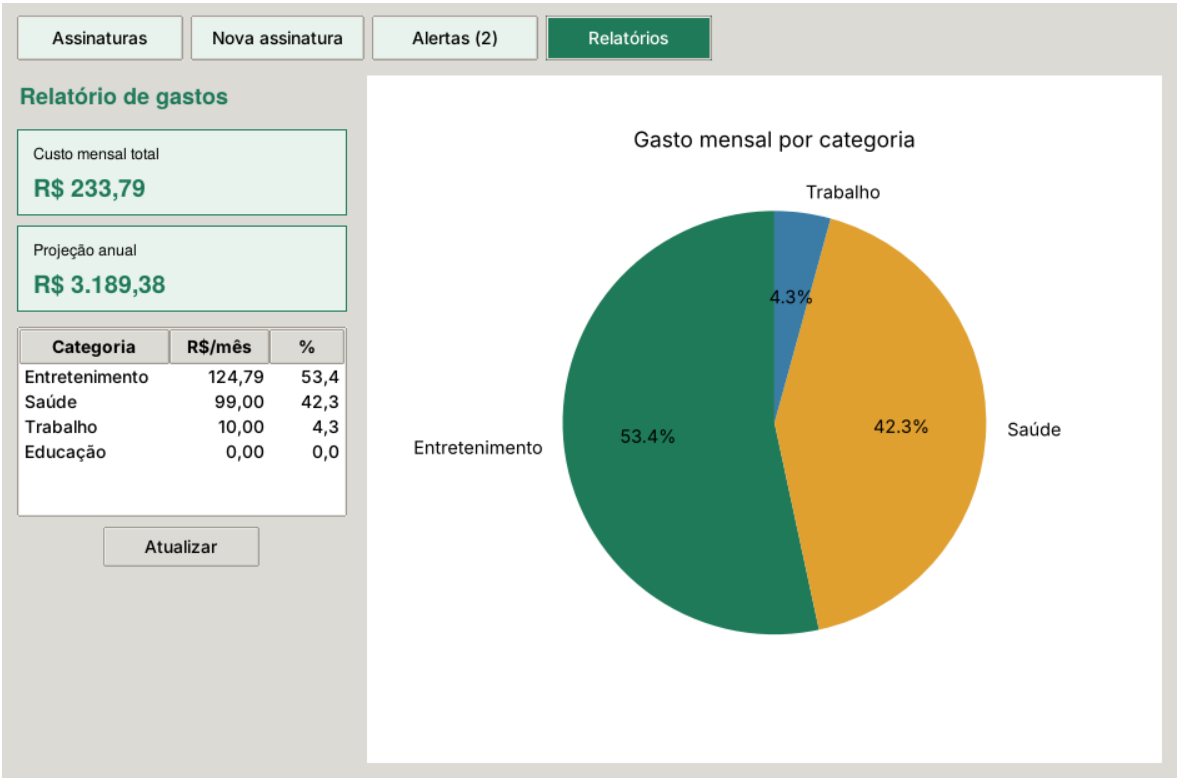


Figura 5 – Relatório de gastos. Totais, tabela de gasto por categoria e gráfico de pizza feito com matplotlib e embutido na janela. Os valores seguem os mesmos dados da Figura 1.

4. Forma de interação com o usuário

A interação é gráfica, por mouse e teclado (a tecla Tab navega entre os campos dos formulários). A barra de abas no topo leva direto de qualquer tela para qualquer outra. Os dados são salvos em *assinaturas.json* a cada alteração.

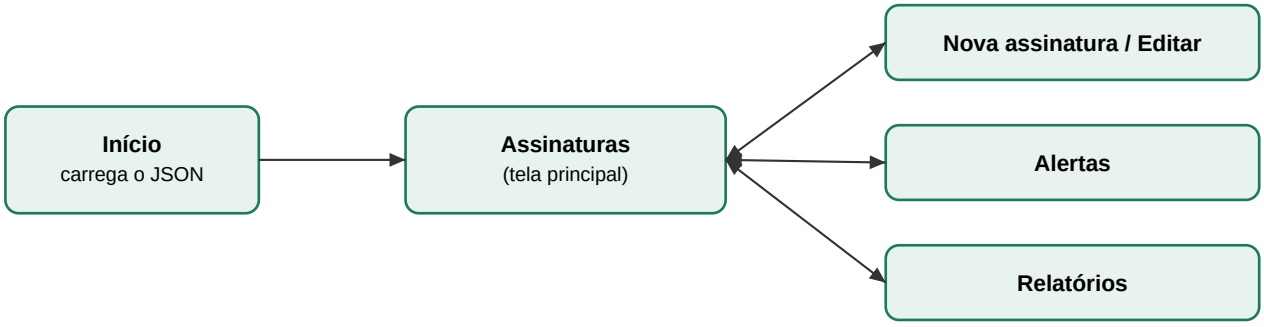


Figura 6 – Navegação entre as telas.

Fluxos principais

Ação do usuário	Resposta do sistema
Abrir o programa	Carrega o *assinaturas.json* (se existir) e mostra a tela principal, com o número de alertas na aba. Se o arquivo estiver corrompido, guarda uma cópia (.bak), avisa e começa vazio.
Clicar em “Carregar exemplos” (lista vazia)	Cadastra 6 assinaturas de demonstração, com datas calculadas a partir de hoje.
Clicar em “Nova assinatura”	Abre o formulário vazio; escolher o tipo exibe os campos específicos.
Preencher e clicar em Salvar	Valida os campos. Se estiver tudo certo, cria o objeto do tipo escolhido, grava o JSON e volta para a lista; se não, mostra a mensagem de erro e mantém o formulário.
Selecionar uma linha e clicar em Editar	Abre o formulário preenchido; ao salvar, atualiza a assinatura.
Selecionar uma linha e clicar em Remover	Pede confirmação; se confirmada, remove a assinatura (e as cobranças dela) e atualiza lista e totais.
Selecionar uma linha e clicar em Registrar pagamento	Cria uma Cobranca com a data de hoje e o valor da cobrança e avança a próxima cobrança. Fica desativado durante um período de teste.
Abrir a aba Alertas	Lista vencimentos e fins de teste dos próximos 7 dias.
Abrir a aba Relatórios	Mostra custo mensal total, projeção anual, gasto por categoria e o gráfico.

Validações do formulário

Campo	Regra	Mensagem de erro
Nome	Obrigatório e único (sem diferenciar maiúsculas).	“Informe um nome.” / “Já existe uma assinatura com esse nome.”
Valor	Número maior que zero (aceita vírgula ou ponto). No período de teste, fixo em 0,00.	“Informe um valor numérico maior que zero.”
Categoria	Uma das categorias da lista.	“Escolha uma categoria.”
Data de início	Formato dd/mm/aaaa e data válida.	“Data inválida. Use dd/mm/aaaa.”
Duração do teste	Número inteiro maior que zero.	“A duração deve ser um número inteiro maior que zero.”
Valor após o teste	Número maior que zero.	“Informe o valor cobrado após o teste.”

5. Repositório do código

Repositório remoto no GitHub: https://github.com/viniciusqmachado/NoStress

Arquivos: *main.py* (programa completo: domínio, interface e programa principal), *testes.py* (testes automáticos) e *README.md* (apresentação do projeto e instruções de uso). O arquivo *assinaturas.json* é criado sozinho na primeira vez que os dados são salvos.

Como executar

- Clonar o repositório: `git clone https://github.com/viniciusqmachado/NoStress.git`
- Instalar a biblioteca do gráfico: `pip install matplotlib`
- Abrir o programa: `python main.py`
- Rodar os testes: `python -m unittest testes -v`

6. Bibliotecas e classes externas

Biblioteca	Para que é usada
tkinter / ttk	Interface gráfica: janela, abas, tabela (Treeview), formulários, botões e caixas de mensagem. Já vem com o Python.
matplotlib	Gráfico de pizza do relatório (FigureCanvasTkAgg embute a figura na janela do Tkinter). Se não estiver instalada, o programa funciona sem o gráfico.
abc	Classes abstratas: Assinatura e TelaBase não podem ser instanciadas diretamente (ABC e @abstractmethod).
datetime e calendar	Datas de início, vencimentos, fim de teste e ajuste do dia no fim do mês (ex.: 31/01 + 1 mês = 28/02).
json, os e shutil	Gravar e ler o arquivo assinaturas.json (gravação por arquivo temporário, para não perder dados se algo falhar) e guardar a cópia .bak quando o arquivo está corrompido.
unittest	Testes automáticos do domínio.

Também herdamos de classes do Tkinter: *JanelaPrincipal* herda de *tk.Tk* e *TelaBase* herda de *ttk.Frame*.

7. CRUD

O CRUD é feito sobre as assinaturas, pela tela principal e pelo formulário. Cada operação grava o JSON em seguida.

Operação	Na interface	No código
Create	Nova assinatura > Salvar; Registrar pagamento (cria uma Cobranca)	GerenciadorAssinaturas.adicionar(), registrar_pagamento()
Read	Tela principal, Alertas e Relatórios	listar(), buscar(), gerar_alertas(), classe Relatorio
Update	Editar > Salvar	substituir() (mantém o histórico de pagamentos)
Delete	Remover (com confirmação)	remover() (apaga também as cobranças da assinatura)

As categorias são uma lista fixa, sem CRUD próprio.

8. Exemplos de testes

Testes automáticos (27). Cobrem polimorfismo, encapsulamento, próxima cobrança (incluindo fim de mês e pagamento antecipado), gerenciador (duplicados, edição, remoção), alertas, relatório, persistência em JSON e as funções auxiliares. Saída de `python -m unittest testes -v`:

```
test_validacoes_do_construtor (testes.TesteEncapsulamento.test_validacoes_do_construtor) ... ok
test_valor_e_privado (testes.TesteEncapsulamento.test_valor_e_privado) ... ok
test_valor_invalido_e_recusado (testes.TesteEncapsulamento.test_valor_invalido_e_recusado) ... ok
test_ler_data (testes.TesteFuncoesAuxiliares.test_ler_data) ... ok
test_ler_valor (testes.TesteFuncoesAuxiliares.test_ler_valor) ... ok
test_somar_meses_e_formato (testes.TesteFuncoesAuxiliares.test_somar_meses_e_formato) ... ok
test_alertas_dos_proximos_7_dias (testes.TesteGerenciador.test_alertas_dos_proximos_7_dias) ... ok
test_edicao_mantem_historico (testes.TesteGerenciador.test_edicao_mantem_historico) ... ok
test_exemplos_nao_duplicam (testes.TesteGerenciador.test_exemplos_nao_duplicam) ... ok
test_nome_duplicado_e_recusado (testes.TesteGerenciador.test_nome_duplicado_e_recusado) ... ok
test_registrar_pagamento_cria_cobranca (testes.TesteGerenciador.test_registrar_pagamento_cria_cobranca) ... ok
test_remover_apaga_cobrancas (testes.TesteGerenciador.test_remover_apaga_cobrancas) ... ok
test_arquivo_corrompido_vira_backup (testes.TestePersistencia.test_arquivo_corrompido_vira_backup) ... ok
test_arquivo_inexistente_comeca_vazio (testes.TestePersistencia.test_arquivo_inexistente_comeca_vazio) ... ok
test_json_ida_e_volta (testes.TestePersistencia.test_json_ida_e_volta) ... ok
test_assinatura_e_abstrata (testes.TestePolimorfismo.test_assinatura_e_abstrata) ... ok
test_custo_mensal_de_cada_tipo (testes.TestePolimorfismo.test_custo_mensal_de_cada_tipo) ... ok
test_teste_passa_a_cobrar_depois_do_fim (testes.TestePolimorfismo.test_teste_passa_a_cobrar_depois_do_fim) ... ok
test_anual (testes.TesteProximaCobranca.test_anual) ... ok
test_cobranca_hoje_conta_como_proxima (testes.TesteProximaCobranca.test_cobranca_hoje_conta_como_proxima) ... ok
test_mensal_e_ajuste_de_fim_de_mes (testes.TesteProximaCobranca.test_mensal_e_ajuste_de_fim_de_mes) ... ok
test_pagamento_antecipado_quita_o_vencimento_pendente
    (testes.TesteProximaCobranca.test_pagamento_antecipado_quita_o_vencimento_pendente) ... ok
test_pagamento_avanca_a_proxima_cobranca (testes.TesteProximaCobranca.test_pagamento_avanca_a_proxima_cobranca) ... ok
test_periodo_de_teste (testes.TesteProximaCobranca.test_periodo_de_teste) ... ok
test_projecao_anual_simples (testes.TesteRelatorio.test_projecao_anual_simples) ... ok
test_projecao_considera_fim_do_teste (testes.TesteRelatorio.test_projecao_considera_fim_do_teste) ... ok
test_total_e_por_categoria (testes.TesteRelatorio.test_total_e_por_categoria) ... ok

----------------------------------------------------------------------
Ran 27 tests in 0.005s

OK
```

Cenários testados na interface

Cenário	Resultado esperado	Resultado
Abrir com a lista vazia	Totais zerados e botão “Carregar exemplos” ativo.	OK
Carregar exemplos	6 assinaturas, 2 alertas na aba, linhas em amarelo.	OK
Cadastrar com valor “abc”	Mensagem “Informe um valor numérico maior que zero.”	OK
Cadastrar teste com duração 0	Mensagem sobre a duração; formulário mantido.	OK
Corrigir e salvar	Assinatura aparece na lista; arquivo JSON atualizado.	OK
Cadastrar nome repetido (“netflix”)	Mensagem “Já existe uma assinatura com esse nome.”	OK
Editar o valor do Spotify	Lista atualizada com o novo valor.	OK
Registrar pagamento da Netflix	Cria cobrança e avança a próxima data.	OK
Selecionar um teste ativo	Botão “Registrar pagamento” desativado.	OK
Remover com confirmação	Assinatura some da lista e do JSON.	OK
Fechar e reabrir (recarregar o JSON)	Mesmas assinaturas e cobranças.	OK

9. Limitações e bugs conhecidos

- **Atrasos não são controlados.** Se um vencimento passa sem pagamento registrado, o sistema considera o próximo ciclo e não marca a assinatura como atrasada.
- **Cobranças são manuais.** O sistema calcula a próxima data, mas só cria a cobrança quando o usuário clica em “Registrar pagamento”.
- **Alertas** só aparecem com o programa aberto (sem notificação em segundo plano nem e-mail). A janela é fixa em 7 dias.

- A projeção anual é por competência, não fluxo de caixa: uma assinatura anual entra como 1/12 por mês.
- Categorias fixas (sem cadastro de novas pelo programa).
- Sem controle de usuários nem de moedas; os dados ficam em um arquivo local, sem senha.
- Datas dos exemplos são aproximadas: em fins de mês podem variar em um ou dois dias.
- Testes automáticos cobrem só o domínio; a interface foi testada em cenários simulados e de forma manual.
- Nenhum bug conhecido até o momento nos cenários testados.

10. O que falta fazer (até a entrega final, 19/11)

- Preparar os slides da apresentação e ensaiar a demonstração.
- Testar o programa de ponta a ponta em outro computador (instalação do matplotlib).
- Revisar o visual da interface e as mensagens ao usuário.
- Se sobrar tempo: controle de atrasos, categorias editáveis e exportação do relatório.